abstracta

Proceso de selección · Developer

Desafío técnico

SmartLog Analyzer

Hola,

Gracias por tu interés en sumarte a Abstracta. Este desafío es el siguiente paso del proceso. Está pensado para que nos muestres cómo pensás y cómo construís, en condiciones parecidas a las de un proyecto real.

Plazo
Hasta 5 días corridos desde que recibís el desafío.

Dedicación estimada
Entre 10 y 15 horas. Si no llegás con todo, priorizá y contanos qué dejaste afuera.

Tecnología
Lenguaje, framework y base de datos: los que elijas.

Entrega
A través de PIA.

Cómo sigue el proceso

1. **Desafío técnico.** Lo resolvés a tu ritmo y lo subís a PIA.
2. **Conversación con PIA.** Hasta 15 minutos sobre tu solución: tus decisiones, las alternativas que consideraste y lo que harías distinto.
3. **Conversación con el equipo.** 45 minutos con personas del equipo técnico y de People. Recorremos tu solución, revisamos juntos un fragmento de código y trabajamos sobre lo que construiste.

Qué valoramos

- **Criterio.** Qué alternativas consideraste y por qué elegiste la que elegiste.
- **Calidad.** Código que se entiende, se puede probar y se puede cambiar sin miedo.
- **Robustez.** Qué pasa cuando algo falla, no solo cuando todo sale bien.
- **Comunicación.** Que alguien del equipo pueda levantar, entender y continuar tu trabajo sin preguntarte.

No buscamos perfección. Si algo no te sale, documentá qué intentaste y por qué no funcionó: eso también suma.

Sobre el uso de IA

Podés usar las herramientas de IA que prefieras. En Abstracta trabajamos con IA todos los días. Te pedimos dos cosas: que entiendas cada línea que entregás, y que en el README nos cuentes cómo la usaste, qué te aportó y qué decidiste no aceptar.

Decisiones abiertas

Este desafío tiene decisiones abiertas a propósito: no hay una única respuesta correcta. Si algo no está definido, definilo vos y justificalo. Si tenés dudas, escribinos a través de PIA. Las preguntas también nos cuentan cómo pensás.

El desafío

SmartLog Analyzer

Tu equipo necesita una herramienta interna para analizar los logs de varios microservicios. Hoy los logs se revisan a mano y se pierden patrones importantes. Tu misión es construir un prototipo funcional que reciba logs, los almacene y use IA para detectar patrones y generar insights.

01 Arquitectura

Diseñá la solución antes de escribir código.

Se pide

- Un diagrama de arquitectura con los componentes, cómo se comunican y el flujo de datos. El formato es libre.
- Un documento breve, `ARCHITECTURE.md`, con las decisiones técnicas: para cada una, la alternativa que descartaste y por qué.

02 Base de datos

Se pide

- Un modelo de datos para los logs y para los resultados de los análisis.
- Scripts de migración y seed con datos de ejemplo que permitan probar la aplicación.
- Al menos una consulta de agregación, por ejemplo por servicio, nivel o rango de fechas.

03 API REST

Endpoints mínimos

Método	Ruta	Descripción
`POST`	`/api/logs`	Ingestar un log
`GET`	`/api/logs`	Listar logs con filtros
`POST`	`/api/logs/analyze`	Enviar un conjunto de logs a la IA para su análisis
`GET`	`/api/analysis/{id}`	Obtener el resultado de un análisis
`GET`	`/api/health`	Verificar el estado del servicio

Se pide

- Validación de los datos de entrada.
- Manejo de errores con códigos HTTP correctos y respuestas consistentes.
- Tests automatizados, unitarios y de integración, que incluyan casos borde y no solo el camino feliz.

04 Integración con IA

Se pide

- Integrar un LLM que, a partir de un conjunto de logs, devuelva un resumen en lenguaje natural, una categorización de los problemas detectados y sugerencias de acción.
- Persistir los resultados de cada análisis.
- Tests de la integración con el LLM simulado que cubran una respuesta válida, una respuesta con formato inesperado y una demora excesiva (timeout).

Sobre el LLM. Podés usar el proveedor que prefieras (OpenAI, Anthropic, Gemini, un modelo local u otro) con tu propia API key, o un mock realista detrás de una interfaz que permita cambiar a la integración real. Si usás un mock, documentá cómo sería la integración real. No necesitás gastar dinero para resolver el desafío.

05 Contenedores y despliegue

Se pide

- Un `Dockerfile` para la aplicación.
- Un `docker-compose.yml` que levante la aplicación y la base de datos con un solo comando.
- Documentación de puertos, red y variables de entorno.
- Un `.env.example` sin valores reales.

06 Extras opcionales

Si te queda tiempo y creés que aportan: pipeline de CI, reverse proxy (por ejemplo, Nginx), Makefile o scripts de automatización, documentación OpenAPI, paginación o logging estructurado. Suman si están bien justificados; no restan si no están.

Entrega

Qué entregar

```
smartlog-analyzer/
├── README.md            Cómo levantar, cómo testear, uso de IA y pendientes
├── ARCHITECTURE.md      Diagrama y decisiones técnicas
├── src/                 Código fuente
├── tests/               Tests
├── db/                  Migraciones y seeds
├── Dockerfile
├── docker-compose.yml
└── .env.example
```

La estructura interna de `src/` y `tests/` la definís vos.

El README tiene que incluir

1. Requisitos y cómo levantar el proyecto.
2. Cómo correr los tests.
3. Cómo usaste IA durante el desarrollo.
4. Qué dejaste afuera y cómo lo resolverías.

Antes de entregar, verificá que

- [ ] `docker compose up` levanta la aplicación y la base de datos sin pasos adicionales.
- [ ] Los tests corren con el comando indicado en el README.
- [ ] No hay API keys ni contraseñas reales en el repositorio.
- [ ] Subís a PIA un ZIP que incluye la carpeta `.git` (queremos ver tu historial de commits), o compartís el link a un repositorio público.

Gracias por el tiempo que le vas a dedicar. Esperamos tu entrega con ganas de conocer cómo pensás y cómo construís. **¡Éxitos! Equipo de Abstracta**